

Operating Systems and System Programming

Skill-6

Aim

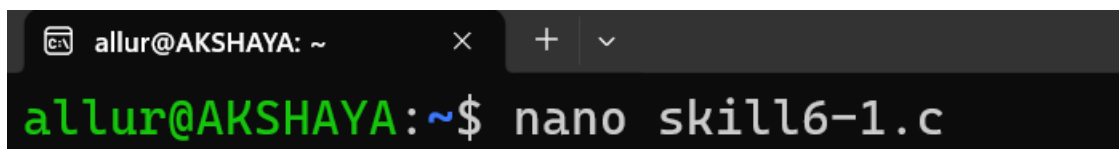
To handle escape sequences, escaped spaces, and special symbols in command input, and to create child processes using `fork()`, execute programs using `exec()`, pass arguments, handle execution errors, and manage parent-child processes.

Algorithm

1. Start the program.
2. Read command input from the user.
3. Identify escape characters and preserve escaped spaces and special symbols.
4. Store and display the parsed output.
5. Create a child process using `fork()`.
6. Check whether the child process is created successfully.
7. Execute the required program using `exec()` and pass arguments.
8. Handle execution errors using `perror()`.
9. Make the parent wait for the child using `wait()`.
10. Display the execution result and terminate the program.

Task-1:

Step-1:

A screenshot of a terminal window. The title bar shows 'allur@AKSHAYA: ~' and standard window controls. The terminal prompt is 'allur@AKSHAYA:~\$' and the command 'nano skill6-1.c' is being entered. The text is displayed in a green monospace font on a black background.

```
allur@AKSHAYA:~$ nano skill6-1.c
```

Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill6-1.c *  
#include <stdio.h>  
#include <string.h>  
int main()  
{  
    char input[100], output[100];  
    int i, j = 0;  
  
    printf("Enter command: ");  
    fgets(input, sizeof(input), stdin);  
  
    for (i = 0; input[i] != '\0' && input[i] != '\n'; i++)  
    {  
        if (input[i] == '\\' && input[i + 1] != '\0')  
            i++;  
  
        output[j++] = input[i];  
    }  
  
    output[j] = '\0';  
  
    printf("Parsed output: %s\n", output);  
    return 0;  
}  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Loc  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go
```

Step-3:

```
allur@AKSHAYA: ~$ gcc skill6-1.c -o skill6-1  
allur@AKSHAYA: ~$ ./skill6-1  
Enter command: hello\ world  
Parsed output: hello world  
allur@AKSHAYA: ~$ ./skill6-1  
Enter command: hello&world  
Parsed output: hello&world  
allur@AKSHAYA: ~$
```

Task-2:

Step-1:

```
allur@AKSHAYA: ~$ nano skill6-2.c
```

Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill6-2.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
int main()  
{  
    pid_t pid = fork();  
    if (pid < 0)  
    {  
        perror("Fork failed");  
        return 1;  
    }  
    if (pid == 0)  
    {  
        printf("Child process executing...\n");  
  
        execlp("ls", "ls", "-l", NULL);  
  
        perror("Exec failed");  
        return 1;  
    }  
    else  
    {  
        printf("Parent waiting for child...\n");  
  
        return 1;  
    }  
}
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

```
    }  
    else  
    {  
        printf("Parent waiting for child...\n");  
        wait(NULL);  
        printf("Child process completed.\n");  
    }  
    return 0;  
}
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

Step-3:

```

allur@AKSHAYA:~$ gcc skill6-2.c -o skill6-2
allur@AKSHAYA:~$ ./skill6-2
Parent waiting for child...
Child process executing...
total 536
-rwxr-xr-x 1 allur allur 16032 Sep 19 15:07 a.out
drwxr-xr-x 2 allur allur 4096 Sep 22 14:35 alarm-clock
-rwxr-xr-x 1 allur allur 36416 Sep 23 18:44 alarm_clock
-rw-r--r-- 1 allur allur 35471 Sep 23 18:44 alarm_clock.c
-rw-r--r-- 1 allur allur 35268 Sep 23 18:57 alarm_clock.c.save
prw-r--r-- 1 allur allur 0 Sep 23 18:44 alarm_control_fifo
-rw-r--r-- 1 allur allur 50 Sep 23 18:44 alarm_history.txt
drwxr-xr-x 2 allur allur 4096 Sep 24 10:26 alarmclock
-rwxr-xr-x 1 allur allur 16256 Oct 1 14:51 client
-rw-r--r-- 1 allur allur 442 Oct 1 14:50 client.c
-rw-r--r-- 1 allur allur 1004 Jul 30 11:01 command
-rw----- 1 allur allur 1004 Jul 30 10:57 command.save
-rw----- 1 allur allur 1004 Jul 30 10:58 command.save.1
-rw----- 1 allur allur 1004 Jul 30 10:59 command.save.2
-rw-r--r-- 1 allur allur 1004 Jul 30 11:02 command_exec.c
-rw----- 1 allur allur 1004 Jul 30 11:00 command_exec.c.save
-rw-r--r-- 1 allur allur 43 Aug 22 13:08 destination.txt
-rw-r--r-- 1 allur allur 173 Aug 23 12:09 exec.c
-rwxr-xr-x 1 allur allur 16032 Aug 23 12:15 fork
-rw-r--r-- 1 allur allur 157 Aug 11 19:58 fork.c
-rwxr-xr-x 1 allur allur 16256 Aug 18 11:22 fork_demo
-rw-r--r-- 1 allur allur 818 Aug 18 11:38 fork_demo.c
-rw-r--r-- 1 allur allur 348 Sep 19 15:07 prac-4.c
-rw-r--r-- 1 allur allur 623 Aug 22 13:07 practical-2.c
-rw-r--r-- 1 allur allur 336 Sep 19 14:49 practical-4
-rw-r--r-- 1 allur allur 531 Sep 19 14:39 practical-4.c
-rw-r--r-- 1 allur allur 815 Aug 22 20:23 practical3.c

```

Result

Escape sequences and special characters were successfully handled, preserving escaped spaces and symbols. Child processes were created and programs were executed using `fork()` and `exec()`, with arguments passed, errors handled, and parent-child synchronization performed successfully.